# Project 2: Inside the Pipeline

EECE.4821/5821 Computer Architecture

Released: **10/28/2026** Due: **11/11/2026, 11:59 pm** (two weeks)

Individual assignment 100 points (4821 and 5821)

## The idea

In Project 1 every instruction took a fixed number of cycles from a table. Real processors overlap instructions in a pipeline, and the number of cycles a program takes depends on how its instructions interact. In this project you take the 5-stage RISC-V pipeline apart one piece at a time in the Ripes simulator. Each part turns on one more piece of hardware: first the control unit, then the pipeline registers, the hazard detection unit, the forwarding unit, and finally branch handling. For each one you will *predict* what should happen, *observe* it in Ripes, and *explain* the difference.

### Learning objectives

1. Explain what each control signal in the single-cycle datapath does, and trace its value for each instruction type.
2. Read a pipeline diagram: say which instruction is in which stage in any cycle, and which pipeline register carries which signal.
3. Identify data hazards and fix them three ways: with `nop`s, with stalls from a hazard detection unit, and with forwarding.
4. Explain why a load-use hazard still needs a stall when forwarding is present, and why a taken branch costs flush cycles.
5. Predict the cycle count of a loop with a formula, and reschedule code to make it faster on a real pipeline.

## 1 Getting started with Ripes

Ripes is a free visual simulator for RISC-V pipelines. Use it in the browser at `https://ripes.me`, or download the desktop app (v2.2.6) from `https://github.com/mortbopet/Ripes/releases`. There is nothing to install for the web version. All expected cycle counts in this handout were measured on v2.2.6.

**Processor selection**
Click the processor icon (top left of the Processor tab) to choose one of the five models below. Each part tells you which one to use.

**Editor**
Paste or open a `.s` file. Ripes assembles it as you type and shows any errors in the bottom panel.

**Step / Run / Reset**
Step one clock cycle at a time, or run to the end. The Processor tab shows **Cycles**, **Instructions retired** and **CPI**.

**Signals**
Hover over any wire or port in the datapath to see its name and current value. Turn on signal values in the display options to label every wire.

**Pipeline diagram**
After running, open the pipeline diagram (table icon in the Processor tab). Cycles are numbered from 0. A stall shows as `-`.

**Registers**
The Registers panel shows every register after the run. Compare it with the single-cycle result to check correctness.

| Ripes model | CLI name | Forwarding | Hazard detection | Used in |
|---|---|---|---|---|
| Single-cycle | `RV32_SS` | n/a | n/a | A |
| 5-stage w/o forwarding or hazard detection | `RV32_5S_NO_FW_HZ` | no | no | B |
| 5-stage w/o forwarding unit | `RV32_5S_NO_FW` | no | yes (stalls) | C |
| 5-stage w/o hazard detection | `RV32_5S_NO_HZ` | yes | no | D |
| 5-stage processor | `RV32_5S` | yes | yes | D, E, F |

**Ripes details that matter for this project**

- `la` is a pseudo-instruction that becomes two instructions (`auipc` + `addi`). Both count toward your cycles.
- The assembler rejects a comment that has an unmatched `(` or `)` on one line, and a label with no instruction after it.
- Write `jalr x0, x1, 0`, not `jalr x0, 0(x1)`.
- The program ends when the PC runs past the last instruction. Do not add an `exit ecall`: it changes the cycle counts.

## 2 Your starter pack

| | |
|---|---|
| `a_control.s` | Part A: a short program that uses every instruction type. |
| `b_hazard.s` | Parts B, C and D: 8 dependent ALU instructions, no loads or branches. |
| `d_load.s` | Parts C and D: loads followed by their uses. |
| `e_loop.s` | Part E: a loop that sums the first `n` words of an array. |
| `f_kernel.s` | Part F: the code you will optimize. |
| `answers.txt` | Template for every numeric and short answer. **Keep the key names.** |

## 3 Part A: The single-cycle datapath and its control 12 pts

Model: `RV32_SS`. File: `a_control.s`.

Every pipeline in this project is the single-cycle datapath cut into stages, so start by learning what each control signal does. Step through `a_control.s` one instruction at a time. For each instruction below, hover over the control unit's outputs and record the values.

**A1. (8 pts)** Fill in this table. In `answers.txt` each row is one line (`A1_lw`, `A1_sub`, ...). Use the value names Ripes shows, such as `REG1/PC`, `REG2/IMM`, and `ALURES/MEMREAD/PC4`. Write `X` where the signal does not matter.

| Instruction | reg_do_ write_ctrl | alu_op1_ ctrl | alu_op2_ ctrl | alu_ ctrl | mem_do_ write_ctrl | reg_wr_ src_ctrl | Branch or jump taken? |
|---|---|---|---|---|---|---|---|
| `lw x5, 0(x20)` | | | | | | | |
| `sub x7, x5, x6` | | | | | | | |
| `beq x7, x0, equal` | | | | | | | |
| `sw x6, 4(x20)` | | | | | | | |
| `jal x1, func` | | | | | | | |
| `slli x10, x5, 1` | | | | | | | |
| `jalr x0, x1, 0` | | | | | | | |
| `beq x9, x0, equal` | | | | | | | |

**A2. (2 pts, hand-graded)** For the taken `beq`, the ALU is not comparing `x7` and `x0`. What is the ALU computing, and which unit makes the branch decision instead? (`A2`, one sentence.)

**A3. (2 pts)** Run the whole program and record cycles and CPI (`A3_cycles`, `A3_cpi`). Then assume these stage delays: IF 200 ps, ID 150 ps, EX 200 ps, MEM 250 ps, WB 100 ps, and 20 ps per pipeline register. Give the single-cycle clock period, the 5-stage clock period, and this program's single-cycle execution time (`A3_tclk_ss`, `A3_tclk_5s`, `A3_time_ss_ps`). You will finish the 5-stage execution time in D5, after you measure its cycle count.

## 4 Part B: Pipeline registers, with no hazard protection 18 pts

Model: `RV32_5S_NO_FW_HZ`. File: `b_hazard.s`.

This pipeline has only the four pipeline registers. Nothing checks for dependences, so it will happily compute the wrong answer.

```
addi x5,  x0, 10
addi x6,  x0, 3
add  x7,  x5, x6
sub  x8,  x7, x5
slli x9,  x8, 2
addi x10, x0, 1
or   x11, x9, x10
add  x12, x11, x7
```

**B1. (3 pts)** Run `b_hazard.s`. It retires 8 instructions in how many cycles (`B1_cycles`)? Explain the extra cycles in one sentence (`B1_why`).

**B2. (3 pts)** From the pipeline diagram, which instruction is in each stage in cycle 4? (`B2_IF`, `B2_ID`, `B2_EX`, `B2_MEM`, `B2_WB`.)

**B3. (4 pts)** Each control signal from Part A is produced in ID, then travels down the pipeline registers until the stage that uses it. For each signal, give the *last* pipeline register it passes through: IF/ID, ID/EX, EX/MEM, or MEM/WB (`B3_<signal>`). Signals: `alu_ctrl`, `alu_op2_ctrl`, `mem_do_write_ctrl`, `reg_wr_src_ctrl`, `reg_do_write_ctrl`. Why must the destination register number `rd` travel all the way to WB too (`B3_rd`)?

**B4. (3 pts)** Compare your final registers with the single-cycle run. List every register from `x5` to `x12` that is wrong (`B4_wrong`). Explain why `x8` ends up as `0xFFFFFFF6` (`B4_x8`).

**B5. (5 pts)** Copy `b_hazard.s` to `b_nops.s` and insert the **fewest** `nop`s that make it correct on `RV32_5S_NO_FW_HZ`. You may not reorder or change the original 8 instructions. *Scoring:* correct `x5`–`x12` (2 pts) and minimal cycles (3 pts). In `B5_rule`, state the rule you used: how many instructions must separate a producer from its consumer in this pipeline, and why that number and not one more?

## 5 Part C: The hazard detection unit 12 pts

Model: `RV32_5S_NO_FW`. Files: the original `b_hazard.s` (no `nop`s) and `d_load.s`.

Now the hardware detects a dependence in ID and stalls until the producer has written the register file. There is still no forwarding.

**C1. (3 pts)** Run the original `b_hazard.s`. Is the result now correct (`C1_correct`)? How many cycles (`C1_cycles`) and how many stall cycles in total (`C1_stalls`)? Compare with your `b_nops.s` from B5 and explain the match in one sentence (`C1_compare`).

**C2. (3 pts)** In the pipeline diagram a stall shows as `-`. For each of the 8 instructions, how many cycles does it wait in ID? Count only the dashes right after its own ID; dashes before its ID belong to the instruction ahead of it (`C2_i1` ... `C2_i8`). Pick one instruction that waits 2 cycles and name the producer it is waiting for (`C2_example`).

**C3. (3 pts, hand-graded)** Step to a cycle where a stall begins and hover over the hazard unit's outputs. Which pipeline registers or the PC are *held* (not updated), and which register is *cleared* to insert a bubble (`C3_held`, `C3_cleared`)? Why must the bubble be inserted into ID/EX rather than simply holding every stage (`C3_why`)?

**C4. (3 pts)** Run `d_load.s` on this model. Record cycles (`C4_cycles`). How many stall cycles does each `lw` → use pair cost here (`C4_loaduse_stalls`)? Keep this number for Part D.

## 6 Part D: The forwarding unit 18 pts

Models: `RV32_5S_NO_HZ` (forwarding only), then `RV32_5S` (forwarding and hazard detection). Files: `b_hazard.s`, `d_load.s`, `a_control.s`.

Forwarding sends a result from the EX/MEM or MEM/WB register straight back to the ALU inputs, so a consumer does not have to wait for the register file.

**D1. (3 pts)** Run the original `b_hazard.s` on `RV32_5S_NO_HZ`. Is it correct without any `nop`s (`D1_correct`), and how many cycles (`D1_cycles`)?

**D2. (5 pts)** Step through `b_hazard.s` and watch the two forwarding muxes in EX, `alu_reg1_forwarding_ctrl` and `alu_reg2_forwarding_ctrl`. For each source operand, where does its value come from: `RF` (the ID/EX register, no forwarding), `EX/MEM`, or `MEM/WB`? Keys are `D2_add7_rs1`, `D2_add7_rs2`, and so on, as listed in `answers.txt`.

| Instruction | rs1 comes from | rs2 comes from |
|---|---|---|
| `add x7, x5, x6` | | |
| `sub x8, x7, x5` | | |
| `slli x9, x8, 2` | | (immediate) |
| `or x11, x9, x10` | | |
| `add x12, x11, x7` | | |

What rule decides between EX/MEM and MEM/WB? When both stages hold a write to the same register, which one must win, and why (`D2_priority`)?

**D3. (4 pts)** Run `d_load.s` on `RV32_5S_NO_HZ`. It is wrong even with forwarding. Copy it to `d_nops.s` and insert the fewest `nop`s that make it correct on this model, without reordering. *Scoring:* correct `x5`–`x12` (2 pts) and minimal cycles (2 pts). In `D3_why`, explain why a `lw` result cannot be forwarded to the very next instruction's EX stage.

**D4. (3 pts)** Run the original `d_load.s` on `RV32_5S`. Record cycles (`D4_cycles`) and stall count (`D4_stalls`). How many stall cycles does each load-use pair cost now (`D4_loaduse_stalls`), compared with your answer to C4?

**D5. (3 pts)** Fill in this table of cycle counts (`D5_<file>_<model>`), and mark whether each result was correct where asked. Which single piece of hardware saved more cycles on `d_load.s`, the forwarding unit or the hazard unit (`D5_which`)? Finally, run `a_control.s` on `RV32_5S` and finish A3: its 5-stage execution time in ps and the speedup over single-cycle (`D5_time_5s_ps`, `D5_speedup`).

| Program | `RV32_SS` | `NO_FW_HZ` (correct?) | `NO_FW` | `NO_HZ` (correct?) | `RV32_5S` |
|---|---|---|---|---|---|
| `b_hazard.s` | | | | | |
| `d_load.s` | | | | | |

## 7 Part E: Control hazards: branches and flushes 15 pts

Model: `RV32_5S`. File: `e_loop.s`, which sums the first `n` words of an array into `x12` (`n` = 8 as given).

The Ripes 5-stage pipeline keeps fetching `PC+4` and resolves branches and jumps in EX. When a branch turns out to be taken, the instructions fetched behind it are flushed.

**E1. (3 pts)** Run `e_loop.s` and look at the first taken `bne` in the pipeline diagram. How many instructions are flushed (`E1_flushed`), and which two are they (`E1_which`)? In which stage is the branch outcome known (`E1_stage`)?

**E2. (2 pts)** Is there a penalty when the `bne` is *not* taken on the last iteration (`E2_nottaken_penalty`)? What does this tell you about the pipeline's built-in branch prediction (`E2_prediction`)?

**E3. (5 pts)** Change `n` to 1, 2, 4 and 8, and record the cycles for each (`E3_n1` … `E3_n8`). Fit a formula cycles($n$) = $a \cdot n + b$ (`E3_a`, `E3_b`). Break $a$ down into instructions, load-use stalls and flush cycles per iteration (`E3_breakdown`). Your formula must predict $n = 16$ exactly; check it.

**E4. (5 pts)** Copy the file to `e_sched.s` and reorder the loop body, without adding or removing instructions, to remove the load-use stall. *Scoring:* graded on Ripes with several values of `n`: correct `x12` (2 pts) and the expected cycles (3 pts). Give your new formula (`E4_a`, `E4_b`). Why can't reordering remove the flush cycles too (`E4_flush`)?

## 8 Part F: The scheduling challenge 15 pts

Model: `RV32_5S`. File: `f_kernel.s`, which computes `a0` = the sum of $|$`arr`$[i]|$ and `a1` = the largest $|$`arr`$[i]|$ over `n` words. As given, it takes **224 cycles** on the 16-word array. Use everything from Parts C–E to make it faster, and submit the result as `f_opt.s`.

**Rules.** RV32I base instructions only (no `mul`, no compressed instructions). Leave the format of the `.data` section unchanged: the grader swaps in other arrays and values of `n`, where `n` is always a multiple of 4 and at least 4. Results must be in `a0` and `a1` when the program runs off the end. Hard-coded answers will fail the hidden arrays.

**F1. (3 pts)** Before you change anything, account for the 224 cycles: how many are instructions, load-use stalls and flush cycles (`F1_instr`, `F1_stalls`, `F1_flush`)? The rest are the 4 pipeline-fill cycles. Which branch in the loop costs the most, and why (`F1_costliest`)?

**F2. (9 pts)** `f_opt.s` must be correct on every hidden array to earn any points here. Points are awarded by the cycle count on the given array:

| Cycles on the given array | $\leq 210$ | $\leq 180$ | $\leq 150$ |
|---|---|---|---|
| Points | 3 | 6 | 9 |

**F3. (3 pts, hand-graded)** In `F3_explain` and in your report, list each change you made and how many cycles it saved.

**Ideas to consider**

Remove a branch with arithmetic; lay out a rarely taken branch so that the common case falls through; unroll the loop; and move loads away from their uses.

## 9 Deliverables and submission

Submit one `<studentLastname>_P2.zip` (for example: `boewe_P2.zip`) containing these files at the top level:

| | |
|---|---|
| `answers.txt` | All parts, with your `username` filled in (autograded) |
| `b_nops.s` | B5 (autograded on `RV32_5S_NO_FW_HZ`) |
| `d_nops.s` | D3 (autograded on `RV32_5S_NO_HZ`) |
| `e_sched.s` | E4 (autograded on `RV32_5S`) |
| `f_opt.s` | F2 (autograded on `RV32_5S`) |
| `report.pdf` | Written report, 2–3 pages. Include one annotated pipeline-diagram screenshot each for a stall (Part C), a forwarding path (Part D) and a flush (Part E). Add the A2, C3 and F3 explanations, and end with a paragraph on which hardware feature mattered most for `f_kernel.s` and why. |

**About the autograder**

Most of the points are computed by a script that runs your `.s` files with the Ripes v2.2.6 command line on the model named in each part. File names and the key names in `answers.txt` must be *exactly* as given. Use numbers only for numeric keys: no units and no formulas. Cycle counts must be exact. CPI and times are accepted within 1 %. A file that fails to assemble scores 0 for its graded runs, so open each one in Ripes before you submit.

## Point summary

| Component | Auto | Hand | Total |
|---|---|---|---|
| A. Single-cycle control | 10 | 2 | 12 |
| B. Pipeline registers, manual `nop`s | 13 | 5 | 18 |
| C. Hazard detection and stalls | 8 | 4 | 12 |
| D. Forwarding | 13 | 5 | 18 |
| E. Branches and flushes | 10 | 5 | 15 |
| F. Scheduling challenge | 12 | 3 | 15 |
| Report quality | 0 | 10 | 10 |
| **Total** | **66** | **34** | **100** |

There is no separate graduate component for this project. EECE.4821 and EECE.5821 students complete the same assignment.